Mexticacán Incidencias

Tu reporte, un mejor municipio

GUÍA PARA RESPONSABLES TÉCNICOS

Manual de mantenimiento

Operación segura, respaldos, actualización y recuperación

SGI · H. Ayuntamiento de Mexticacán, Jalisco

Edición · 15 de octubre de 2026

Alcance. Orientación técnica para mantener la API Node.js, el panel web React/Vite, la app Expo, SQL Server y archivos de imágenes en una instancia controlada.

Los comandos de inicio del repositorio son de desarrollo. La operación en producción debe usar infraestructura administrada, HTTPS, servicios supervisados y un plan municipal de seguridad.

Mapa de mantenimiento

Componente	Responsabilidad	Comprobación
SQL Server · `SGI_Mexticacan`	Usuarios, roles, incidencias, comentarios, historial, notificaciones y rutas de imágenes.	Servicio SQL activo; conexión de API; respaldo vigente.
Backend · `backend`	API Express, JWT, Socket.IO, carga de archivos y acceso a SQL Server.	`GET /api/salud`; revisar salida de Node.js.
Panel web · `web`	Interfaz municipal React/Vite y comunicación con API.	La URL web carga y consulta datos de API.
App móvil · `mobile`	Expo Router, app ciudadana, GPS/cámara y conexión a API.	Expo Doctor/export en entorno de validación; prueba controlada en dispositivo.
Imágenes · `backend/uploads`	Fotos de reportes y evidencias; la base guarda las rutas.	Espacio libre, permisos y respaldo conjunto con la base.

Principios de mantenimiento

- Realiza cambios primero en desarrollo/validación; acuerda ventana de mantenimiento.
- Antes de un cambio, confirma respaldo recuperable de base y `backend/uploads`.
- Conserva los archivos `.env` fuera del control de versiones y con permisos restringidos.
- La respuesta de `/api/salud` verifica HTTP, no demuestra por sí sola el estado correcto de SQL, fotos o inicio de sesión.
- Registra responsable, cambio, fecha, validaciones y resultado de recuperación.

> **Importante:** los scripts `database\01_esquema.sql`, `02_catalogos.sql` y `03_procedimientos.sql` son de inicialización. No los ejecutes indiscriminadamente sobre una base existente; realiza cambios de esquema mediante un procedimiento versionado y respaldado.

Verificación y ciclo de servicio

Revisión de inicio de jornada

1. Confirma que SQL Server está activo y que el equipo/servicio puede conectarse a la base SGI.
2. Confirma la API con `http://servidor:4000/api/salud` y revisa que no existan errores de conexión en el proceso.
3. Abre panel web, inicia sesión con una cuenta de prueba autorizada y revisa incidencias, notificaciones y mapa.
4. En una prueba móvil controlada, confirma que Expo y la API son accesibles desde el teléfono; no uses localhost en el teléfono.
5. Revisa espacio libre para base, logs y `backend\uploads`.

Inicio local de desarrollo (PowerShell)

Abre tres terminales en la raíz. SQL Server debe estar activo y cada `.env` correctamente configurado.

```
# Terminal 1: API
Set-Location .\backend
npm install
npm run dev

# Terminal 2: panel web
Set-Location .\web
npm install
npm run dev

# Terminal 3: app Expo
Set-Location .\mobile
npm install
npx expo start
```

Para teléfono, configura `EXPO_PUBLIC_API_URL` con la IP local de la computadora y puerto 4000; no uses `localhost`. Para detener el desarrollo, `Ctrl+C` en cada terminal.

> **Desarrollo no es producción:** `npm run dev` activa el modo de observación de Node y Vite; Expo inicia Metro. No son, por sí solos, una topología de despliegue productivo.

Seguimiento de eventos

El backend escribe mensajes en stdout/stderr. En producción, el responsable debe redirigirlos a un recolector con rotación y retención. No guardar contraseñas ni tokens en logs o capturas.

Variables, cuentas y secretos

Archivo privado	Variables/ajustes relevantes	Regla operativa
`backend\.env`	`PORT`, `DB_SERVER`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_ENCRYPT`, `DB_TRUST_CERT`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `CORS_ORIGIN`, `MAX_IMAGE_MB`.	Conservar en el servidor; restringir permisos. `JWT_SECRET` largo, aleatorio y confidencial.
`web\.env`	`VITE_API_URL`.	Apuntar a la API correcta. No poner secretos privados en variables expuestas al navegador.
`mobile\.env`	`EXPO_PUBLIC_API_URL`.	URL accesible desde el dispositivo; cualquier variable `EXPO_PUBLIC_` es pública en el cliente.

Rutina de seguridad

- Restringe SQL Server, carpetas de cargas, `.env`, respaldos y exportaciones a personal autorizado.
- Restringe `CORS_ORIGIN` a orígenes municipales confiables en despliegues accesibles fuera del equipo local.
- No expongas HTTP de desarrollo a Internet. Usa HTTPS y controles de red/identidad de la infraestructura oficial.
- Administra usuarios con cuentas individuales; desactiva accesos al terminar la necesidad y aplica privilegio mínimo.
- Al rotar `JWT_SECRET`, planifica que las sesiones firmadas con el valor anterior serán invalidadas; comunica un nuevo inicio de sesión.
- Las fotos contienen información municipal; evita copiarlas a destinos personales o publicar enlaces.

Subidas de imágenes

El backend guarda archivos bajo `backend\uploads`; los nombres se generan con identificador único. Se admiten JPG, PNG, WEBP y HEIC, máximo 5 archivos por carga, con tamaño por archivo gobernado por `MAX_IMAGE_MB` (por defecto 8 MB). Debe conservarse la carpeta junto con la base para que las referencias existentes sigan funcionando.

> **Credenciales comprometidas:** desactiva la cuenta afectada si corresponde, cambia secreto/contraseña con autorización, revisa accesos y conserva evidencia para investigación. No borres logs o registros pertinentes.

4 · RESPALDOS

Respaldar y validar recuperación

El repositorio no instala ni programa un servicio automático de respaldos. El responsable técnico municipal debe establecer frecuencia, retención, cifrado, ubicación, custodios y pruebas de restauración.

Conjunto mínimo a proteger

- Base de datos SQL Server `SGI_Mexticacan`.
- Archivos de `backend\uploads`.
- Configuración privada `.env` mediante un gestor seguro de secretos (no copiarla al repositorio junto con el respaldo).
- Versión de código/dependencias y scripts de base de datos usados para esa versión.

Ejemplo de respaldo manual de SQL Server

Ejecuta en SSMS con una cuenta autorizada y una ruta protegida existente en el servidor de base de datos. Cambia ruta y nombre según la política local:

```
BACKUP DATABASE [SGI_Mexticacan]
TO DISK = N'D:\Respaldos\SGI_Mexticacan_full.bak'
WITH COPY_ONLY, CHECKSUM, COMPRESSION, INIT, STATS = 10;

RESTORE VERIFYONLY
FROM DISK = N'D:\Respaldos\SGI_Mexticacan_full.bak'
WITH CHECKSUM;
```

Si la edición de SQL Server no admite una opción, confirma el comando con el DBA antes de quitarla. Protege la ruta; `INIT` reemplaza el archivo de respaldo indicado.

Respaldo de archivos y consistencia

La base contiene rutas de fotografías, no los bytes de imagen. Copia `backend\uploads` a almacenamiento protegido. Coordina las copias con un snapshot consistente o pausa temporalmente escrituras/cargas para evitar que base y archivos representen momentos incompatibles. Verifica conteo/tamaño, permisos y restauración de archivos.

> **Restauración:** prueba en una instancia aislada con nombre y rutas separadas. No uses `WITH REPLACE` ni restaures sobre producción durante una prueba. Verifica inicio de sesión, lectura de registros, detalle e imágenes de una muestra autorizada.

5 · ACTUALIZACIONES

Aplicar cambios con control

1. Registra solicitud, motivo, componentes afectados, responsable, ventana y criterio de reversión.
2. Genera y valida respaldo de SQL Server y `backend\uploads` antes de actualizar.
3. Obtén la versión aprobada del código y conserva `package-lock.json`; no alteres versiones sin revisar compatibilidad.
4. Instala desde lockfile en cada proyecto usando la versión de Node/npm aprobada.
5. Ejecuta pruebas/build/Expo Doctor en un entorno de desarrollo antes de habilitar usuarios.
6. Si el cambio modifica esquema, prepara y revisa un script incremental reversible; no vuelvas a ejecutar los scripts de creación inicial como migración.
7. Aplica la actualización en la ventana acordada y realiza verificaciones funcionales.
8. Documenta resultado, incidencias y decisión de mantener o revertir.

Validación del repositorio

```
# API: pruebas (solo base de desarrollo)
Set-Location .\backend
npm ci
npm test

# Web: compilación
Set-Location ..\web
npm ci
npm run build

# App móvil: verificaciones y bundle Android
Set-Location ..\mobile
npm ci
npx expo-doctor
npx expo export --platform android
```

> **Advertencia de pruebas:** las pruebas de backend crean datos temporales y limpian esa corrida. Ejecuta `npm test` exclusivamente contra una base de desarrollo aislada; nunca contra producción.

Para que el usuario móvil reciba el nuevo bundle Expo, sigue el proceso de distribución configurado por el equipo. `expo export` valida/exporta, no publica por sí solo en tiendas ni actualiza automáticamente dispositivos.

Respuesta ante fallas frecuentes

Problema	Diagnóstico inicial	Acción segura
API sin conexión	Revisa stdout/stderr y endpoint de salud.	Confirma proceso, puerto, variable `PORT` y conectividad a SQL; conserva el error sin secretos.
Error SQL	Valida servicio, servidor/puerto/base, permisos y TCP/IP.	No cambies ni recrees tablas a ciegas. Consulta al DBA y conserva respaldo antes de cualquier ajuste.
Web no carga datos	Verifica URL de web y `VITE_API_URL`; revisa API/CORS.	Confirma origen permitido y que la API responde desde la red del panel.
Móvil no conecta	Valida IP, `EXPO_PUBLIC_API_URL`, Metro, Wi-Fi y firewall.	Usa dirección de servidor alcanzable; reinicia bundle de desarrollo después de editar configuración.
Imagen no aparece	Revisa la ruta DB, archivo bajo `backend\uploads`, permisos y espacio.	Restaura desde copia aprobada; no borres archivos para liberar espacio sin identificar referencias.
Disco lleno	Revisa base, uploads, logs y backups.	Escala al responsable; no elimines logs, evidencia o copias retenidas sin autorización.

Procedimiento de recuperación

1. Notifica a responsables y determina alcance, hora de inicio y servicios afectados.
2. Evita escribir sobre base/archivos si hay corrupción o pérdida sospechada. Conserva logs y copias originales.
3. Decide con DBA/municipio si se restaura una copia en entorno aislado o en el servicio controlado.
4. Restaura base y uploads correspondientes al mismo punto temporal; valida datos, imágenes, login y permisos.
5. Reabre acceso gradualmente y registra el incidente, decisiones, tiempo de recuperación y pendientes.

> **Escalamiento:** fallas con datos personales, acceso indebido o exposición de secretos deben notificarse al responsable municipal de seguridad por el canal institucional.

7 · REGISTRO DE MANTENIMIENTO

Control de cambios y revisión

Completa un registro por cada intervención de mantenimiento:

Campo	Información a registrar
Fecha/hora y responsable	Nombre institucional y periodo de intervención.
Versión antes / después	Hash/etiqueta de código, versiones Node/npm, paquetes y entorno.
Motivo y alcance	Incidencia, actualización o tarea preventiva; componentes afectados.
Respaldo	Identificador y ubicación protegida, verificación, inclusión de uploads y prueba de restauración.
Acciones y validaciones	Pasos, resultados HTTP, pruebas controladas y observaciones; no incluir secretos.
Resultado / reversión	Éxito, pendientes, decisión, responsable de aprobación y ruta de retorno.

Lista periódica recomendada

Diaria: salud de API/SQL, errores, espacio disponible, colas de trabajo y reportes críticos.

Periódica según política municipal: validar que existan respaldos, revisar accesos/cuentas activas, rotación de logs, capacidad de uploads y alertas.

Antes de cada liberación: pruebas en desarrollo, chequeo de roles, cargas de imagen, conexión móvil/web, respaldo y procedimiento de reversión.

Después de cada recuperación: reconciliar cantidad/identidad de registros y accesibilidad de imágenes; confirmar con responsables que el servicio es consistente.

Este manual describe las capacidades visibles del repositorio y recomienda que el Ayuntamiento formalice RPO/RTO, retención, monitoreo, cifrado, responsables y horarios de mantenimiento según sus obligaciones.